

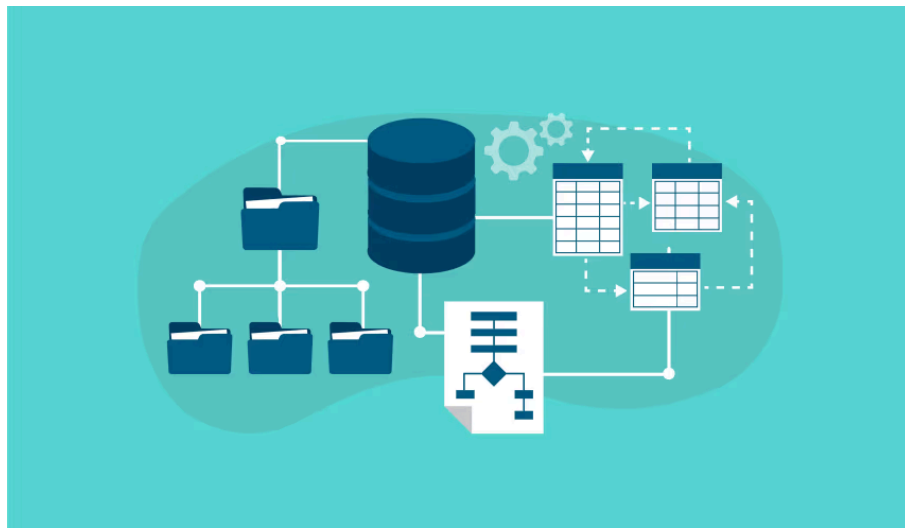
Proyecto Final

Materia: Estructura de Datos

Maestro: Adalberto Emmanuel Rojas Perea

Alumna: Isabel Kyra Romero Borunda

TECMILENIO



Índice

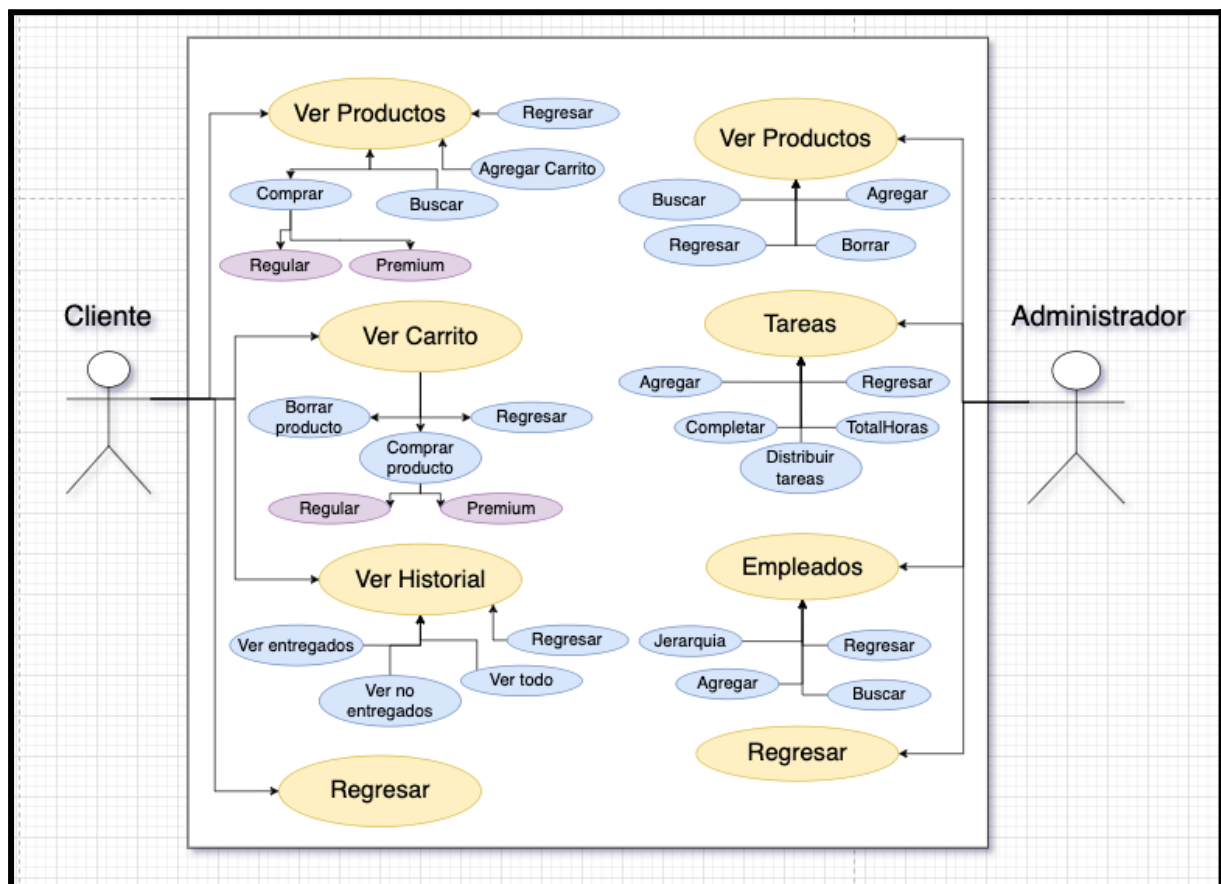
Índice.....	2
Introducción.....	3
Implementación.....	3
Diagrama.....	3
Estructura.....	3
Clases.....	3
Métodos.....	6
Usuarios.....	8
Resultados.....	8
Administrador.....	8
Cliente.....	12
Conclusión.....	14

Introducción

En este proyecto vamos a implementar clases y métodos previamente vistos a nuestro avance de proyecto. Se puso en uso un árbol binario, y se usaron conceptos como la recursividad, la búsqueda y ordenamiento, así como el concepto de divide y vencerás en algoritmos que nos harán más fácil manejar una tienda en línea, por último se sustituyen unas clases y se agregaron otras.

Implementación

Diagrama



Estructura

Se importaron **'ArrayList'**, **'HashMap'**, **'Stack'** sustituyendo los métodos de 'Lista' y 'Stack' previamente hechos en el avance de proyecto para mayor facilidad.

Clases

- Tarea

Esta clase se utilizó para crear tareas, donde este puede contener el 'Nombre', la 'Prioridad' y una 'Descripción'. Contiene tres métodos getters, cada uno para que nos devuelva el atributo que queramos.

```
// Tarea
class Tarea{
    String nombre;
    int prioridad;
    String descripcion;
    int horasEstimadas;
    public Tarea(String nombre, int prioridad, String descripcion, int horasEstimadas){
        this.nombre = nombre;
        this.prioridad = prioridad;
        this.descripcion = descripcion;
        this.horasEstimadas = horasEstimadas;
    }

    // Getters
    public String getNombre() {
        return this.nombre;
    }
    public int getPrioridad() {
        return this.prioridad;
    }
    public String getDescripcion() {
        return this.descripcion;
    }
    public int getHoras() {
        return this.horasEstimadas;
    }
}
```

- **Producto**

Se utiliza para crear productos, cuenta con 3 atributos '**Nombre**', '**Precio**' y '**Entregado**'. El atributo entregado es de tipo booleano, para que cuando se haya entregado su valor sea **True**.

```
// Producto
class Producto{
    String nombre;
    boolean entregado;
    int precio;

    public Producto(String nombre, int precio) {
        this.nombre = nombre;
        this.entregado = false;
        this.precio = precio;
    }

    // Getters

    public String getNombre() {
        return this.nombre;
    }
    public int getPrecio() {
        return this.precio;
    }
}
```

- **PriorityQueue / Cola de prioridad**

Se usó para guardar las tareas según su prioridad. Entran cuando el cliente pide un producto. Tiene cuatro métodos que nos permiten agregar, borrar, ver el primero e imprimir toda la lista. Al igual que su clase nodo se le hicieron pequeños cambios para que pudiera manejar objetos de la clase Tarea.

```
// Cola de Prioridad
class PriorityQueue {
    PriorityNode[] data;
    int tamaño;
    int limite;

    public PriorityQueue() {
        limite = 50;
        data = new PriorityNode[limite + 1];
        tamaño = 0;
    }

    // Enqueue
    public void enqueue(Tarea dato, int priority) {--
    // Dequeue
    public Tarea dequeue() {--
    // Peek
    public Tarea peek() {--
    // Mostrar
    public void mostrar() {--
}
```

- **ArbolBinario**

Se usó para guardar a los empleados jerárquicamente, donde en el primer nodo(raíz) se guarda al Jefe.

```
// Arbol Binario
class ArbolBinario {
    NodoArbol raiz;

    public ArbolBinario() {
        raiz = null;
    }

    // Insertar nodo
    public void agregarNodo(int valor, String nom) {--

    // Buscar nodo
    public NodoArbol buscar(NodoArbol nodo, int id) {--

    public boolean estaVacio() {--

    // Eliminación
    public NodoArbol eliminar(NodoArbol nodo, int valor) {--

    // Encontrar el nodo con valor mínimo en un subárbol
    private NodoArbol encontrarMinimo(NodoArbol nodo) {--

    // Mostrar arbol
    public void mostrar(NodoArbol nodo, String espacio) {--
}
```

Métodos

- **buscarPorNombre (Búsqueda)**

Este método regresa el producto si se encontró, de lo contrario nos retorna null.

```
// BUSQUEDA
public static Producto buscarPorNombre(String nombre, ArrayList<Producto> productos) {
    for (Producto producto : productos) {
        if (producto.getNombre().equalsIgnoreCase(nombre)) {
            return producto;
        }
    }
    return null; // no encontrado
}
```

- **sumarHoras (Recursividad)**

Suma las horas de todas las tareas, para saber cuántas horas de trabajo faltan.

```
// Sumar horas de tareas recursivamente
public static int sumarHoras(List<Tarea> lista, int inicio, int fin) {
    if (inicio > fin) return 0;
    if (inicio == fin) return lista.get(inicio).getHoras();
    int medio = (inicio + fin) / 2;
    return sumarHoras(lista, inicio, medio) + sumarHoras(lista, medio + 1, fin);
}
```

- **distribuirTareas (Divide y vencerás)**

Este método utiliza otros tres métodos(**DistribuirRecHelper**, **empleadoConMenorCarga** y **horasAsignadasEmpleado**) para poder dividir las tareas entre los empleados.

```
// Distribuir tareas entre empleados rec
public static void distribuirTareas(List<Tarea> tareas, List<Integer> empleadosIds, HashMap<Integer, List<Tarea>> asignaciones) {
    if (tareas == null || tareas.isEmpty() || empleadosIds.isEmpty()) return;
    distribuirRecHelper(tareas, inicio:0, tareas.size() - 1, empleadosIds, asignaciones);
}
```

- **DistribuirRecHelper**

Divide la lista de tareas y asigna la tarea al empleado con menos carga, con la ayuda del siguiente método 'empleadoConMenorCarga'.

```
private static void distribuirRecHelper(List<Tarea> tareas, int inicio, int fin, List<Integer> empleadosIds, HashMap<Integer, List<Tarea>> asignaciones) {
    if (inicio > fin) return;
    if (inicio == fin) {
        int emp = empleadoConMenorCarga(empleadosIds, asignaciones);
        asignaciones.putIfAbsent(emp, new ArrayList<>());
        asignaciones.get(emp).add(tareas.get(inicio));
        return;
    }
    int medio = (inicio + fin) / 2;
    distribuirRecHelper(tareas, inicio, medio, empleadosIds, asignaciones);
    distribuirRecHelper(tareas, medio + 1, fin, empleadosIds, asignaciones);
}
```

- **empleadoConMenorCarga**

Nos regresa el ID del empleado con menor carga de trabajo(horas).

```
// Obtener empleado con menor carga de horas
private static int empleadoConMenorCarga(List<Integer> empleadosIds, HashMap<Integer, List<Tarea>> asignaciones) {
    int mejor = empleadosIds.get(index:0);
    int minHoras = horasAsignadasAEmpleado(mejor, asignaciones);
    for (int id : empleadosIds) {
        int horas = horasAsignadasAEmpleado(id, asignaciones);
        if (horas < minHoras) {
            minHoras = horas;
            mejor = id;
        }
    }
    return mejor;
}
```

- **horasAsignadasEmpleado**

Suma todas las horas de los trabajos que tiene un empleado, este le ayuda al método '**empleadoConMenorCarga**' para saber quien tiene menos carga.

```
// Sumar horas asignadas a un empleado
private static int horasAsignadasAEmpleado(int id, HashMap<Integer, List<Tarea>> asignaciones) {
    List<Tarea> l = asignaciones.get(id);
    if (l == null || l.isEmpty()) return 0;
    int sum = 0;
    for (Tarea t : l) sum += t.getHoras();
    return sum;
}
```

- **obtenerIdsEmpleados**

Este recorre el árbol binario en preorden para obtener los IDs de los empleados.

```
private static void obtenerIdsEmpleados(NodoArbol nodo, List<Integer> ids) {
    if (nodo == null) return;
    ids.add(nodo.valor);
    obtenerIdsEmpleados(nodo.hijoizquierdo, ids);
    obtenerIdsEmpleados(nodo.hijoderecho, ids);
}
```

- **mostrarJerarquia (grafos)**

Este método nos ayuda a mostrar la jerarquía de los empleados.

```
// Grafos
public static void mostrarJerarquia(HashMap<Integer, List<Integer>> subordinados, HashMap<Integer, String> nombres, int jefeId, int nivel) {
    // Indentación según nivel
    for (int i = 0; i < nivel; i++) System.out.print(s: " ");
    System.out.println(nombres.get(jefeId) + " (ID: " + jefeId + ")");

    // Recorrer subordinados
    List<Integer> subs = subordinados.get(jefeId);
    if (subs != null) {
        for (int sub : subs) {
            mostrarJerarquia(subordinados, nombres, sub, nivel + 1);
        }
    }
}
```

Usuarios

Administrador

El administrador cuenta con funciones nuevas, como se pudo ver en el diagrama. Se le agregó opciones como las tareas, los empleados y las asignaciones de tareas, también se le agregó el método de buscar productos. El administrador puede agregar, eliminar, distribuir tareas y sacar el total de horas de todos las tareas juntas.

También puede agregar empleados, buscar y verlos de manera jerárquica.

Cliente

El cliente tiene una función nueva en productos, la cual le permite buscar un producto para ver si está o no está en el catálogo. También ahora puede ver dos historiales nuevos, uno de productos no entregados y otro de productos entregados.

Resultados

Administrador

Productos

- Agregar producto: Se agregó un Chocolate como producto.

```
Bienvenido Administrador

Que quiere hacer?
  1-. Ver Productos
  2-. Ver Tareas
  3-. Ver Empleados
  4-. Salir
>>> 1

PRODUCTOS:

- Camisa $45
- Pantalon $60
- Zapatos $100
- Collar $45
- Lentes de sol $55

1-. Agregar producto   2-. Borrar producto   3-. Buscar producto
>>> 1

Ingrese un producto: Chocolate

Ingrese el precio: 34
Producto 'Chocolate' Agregado
```


- Borrar producto: Se borro pantalón de productos.

```
PRODUCTOS:
- Camisa $45
- Pantalon $60
- Zapatos $100
- Collar $45
- Lentes de sol $55
- Chocolate $34
1-. Agregar producto  2-. Borrar producto  3-. Buscar producto
>>> 2

Ingrese el nombre del producto
>>> Pantalon

Producto 'Pantalon' borrado...
```

```
PRODUCTOS:
- Camisa $45
- Zapatos $100
- Collar $45
- Lentes de sol $55
- Chocolate $34
```

- Buscar: Se buscó collar en la lista de producto y se encontró.

```
PRODUCTOS:
- Camisa $45
- Zapatos $100
- Collar $45
- Lentes de sol $55
- Chocolate $34
1-. Agregar producto  2-. Borrar producto  3-. Buscar producto  4-. Regresar
>>> 3

Ingrese el nombre del producto que desea buscar
>>> Collar
Si esta este producto: Collar
```

Tareas

- Agregar tarea: se agregó una tarea 'Reporte'.

```
TAREAS:
Empaquetar pedido 1 (prioridad 2)
Empaquetar pedido 4 (prioridad 2)
Empaquetar pedido 3 (prioridad 4)
Empaquetar pedido 2 (prioridad 3)

1-. Agregar tarea   2-. Completar proxima tarea   3-. Distribuir tareas   4-. Ver horas de trabajo faltantes
>>> 1

Ingrese el nombre de la tarea: Reporte

Ingrese la prioridad de la tarea: 2

Ingrese una descripcion: Reporte empeques

Ingrese las horas estimadas: 3
Tarea 'Reporte' Agregada
```

- Completar tarea: se completó la tarea con más prioridad (Empaquetar pedido 1).

```
TAREAS:
Empaquetar pedido 1 (prioridad 2)
Empaquetar pedido 4 (prioridad 2)
Enviar (prioridad 3)
Empaquetar pedido 2 (prioridad 3)
Reporte (prioridad 2)
Empaquetar pedido 3 (prioridad 4)

1-. Agregar tarea   2-. Completar proxima tarea   3-. Distribuir tareas   4-. Ver horas de trabajo faltantes
>>> 2
Tarea 'Empaquetar pedido 1' completada
```

- Distribuir tareas: podemos ver las tareas distribuidas por empleados.

```
TAREAS:
Empaquetar pedido 4 (prioridad 2)
Reporte (prioridad 2)
Enviar (prioridad 3)
Empaquetar pedido 2 (prioridad 3)
Empaquetar pedido 3 (prioridad 4)

1-. Agregar tarea   2-. Completar proxima tarea   3-. Distribuir tareas   4-. Ver horas de trabajo faltantes
>>> 3

Tareas distribuidas:
Empleado ID 1:
- Empaquetar pedido 4 (4 hrs)
Empleado ID 201:
- Reporte (3 hrs)
Empleado ID 202:
- Empaquetar pedido 2 (2 hrs)
Empleado ID 301:
- Enviar (5 hrs)
Empleado ID 302:
- Empaquetar pedido 3 (4 hrs)
```

Empleados

- Agregar empleado: agregamos a Diego como empleado.

```
EMPLEADOS:
ID: 1 -> Nombre: Isabel
  ID: 201 -> Nombre: Dafne
    ID: 202 -> Nombre: Valentin
      ID: 301 -> Nombre: Jocelyn
        ID: 302 -> Nombre: David

1-. Agregar empleado  2-. Buscar empleado por ID  3-. Mostrar jerarquia(grafos)
>>> 1
Ingrese el Nombre y el ID del empleado:
>>> ID: 401
>>> Nombre: Diego

Que quiere hacer?
  1-. Ver Productos
  2-. Ver Tareas
  3-. Ver Empleados
  4-. Salir
>>> 3

EMPLEADOS:
ID: 1 -> Nombre: Isabel
  ID: 201 -> Nombre: Dafne
    ID: 202 -> Nombre: Valentin
      ID: 301 -> Nombre: Jocelyn
        ID: 302 -> Nombre: David
        ID: 401 -> Nombre: Diego
```

- Buscar empleado: Buscamos al empleado con el ID -> 202.

```
EMPLEADOS:
ID: 1 -> Nombre: Isabel
  ID: 201 -> Nombre: Dafne
    ID: 202 -> Nombre: Valentin
      ID: 301 -> Nombre: Jocelyn
        ID: 302 -> Nombre: David
        ID: 401 -> Nombre: Diego

1-. Agregar empleado  2-. Buscar empleado por ID  3-. Mostrar jerarquia(grafos)  4-. Regresar
>>> 2

Ingrese el ID(año, mes, número actual de empleado: 4) del empleado que quiere buscar:
>>> ID: 202

Empleado encontrado:
ID: 202
Nombre: Valentin
```

- Jerarquía(grafos): aquí nos muestra la jerarquía, donde Isabel vendría siendo la jefa, Dafne y Jocelyn Gerentes y Valentin y David serían empleados.

```
EMPLEADOS:
ID: 1 -> Nombre: Isabel
  ID: 201 -> Nombre: Dafne
    ID: 202 -> Nombre: Valentin
      ID: 301 -> Nombre: Jocelyn
        ID: 302 -> Nombre: David
        ID: 401 -> Nombre: Diego

1-. Agregar empleado  2-. Buscar empleado por ID  3-. Mostrar jerarquia(grafos)  4-. Regresar
>>> 3
Isabel (ID: 1)
  Dafne (ID: 201)
    Valentin (ID: 202)
      Jocelyn (ID: 301)
        David (ID: 302)
```

Cliente

Productos

- Pedir un producto: pedimos una camisa.

```

Bienvenido a la Tienda Online JDGATE

Que quiere hacer?
1-. Ver productos
2-. Ver carrito
3-. Ver historial de compras
4-. Salir
>>> 1

PRODUCTOS:

- Camisa $45
- Pantalon $60
- Zapatos $100
- Collar $45
- Lentes de sol $55

Que desea hacer ?
1-. Comprar producto 2-. Buscar producto 3-. Agregar a carrito 4-. Regresar
>>> 1

Ingrese el nombre del producto deseado
>>> camisa

Producto pedido: Camisa

Que tipo de entrega desea?:
1-. Premium
2-. Regular
(Si elige premium se le dara más prioridad a su pedido al enviar)
>>> 2
Entendido

Gracias por su compra!
```

- Buscar Producto: buscamos el producto collar y si lo hallamos.

```

PRODUCTOS:

- Camisa $45
- Pantalon $60
- Zapatos $100
- Collar $45
- Lentes de sol $55

Que desea hacer ?
1-. Comprar producto 2-. Buscar producto 3-. Agregar a carrito 4-. Regresar
>>> 2

Ingrese el nombre del producto que desea buscar
>>> Collar
Si esta este producto: Collar
```

- Agregar producto a un carrito: agregamos unos zapatos a nuestro carrito

```
PRODUCTOS:
- Camisa $45
- Pantalon $60
- Zapatos $100
- Collar $45
- Lentes de sol $55

Que desea hacer ?
1-. Comprar producto 2-. Buscar producto 3-. Agregar a carrito 4-. Regresar
>>> 3

Ingrese el nombre del producto deseado
>>> Zapatos

Producto agregado al carrito!: Zapatos

Que quiere hacer?
1-. Ver productos
2-. Ver carrito
3-. Ver historial de compras
4-. Salir
>>> 2

CARRITO
- Reloj $100
- Bolsa Roja $70
- Lentes de sol $55
- Zapatos $100
```

Carrito

- Comprar de carrito: compramos un producto desde el carrito.

```
CARRITO
- Reloj $100
- Bolsa Roja $70
- Lentes de sol $55
- Zapatos $100

Desea comprar o borrar algo de su carrito?
1-. Comprar 2-. Borrar 3-. Salir
>>> 1

Ingrese el nombre del producto
>>> Reloj

Producto pedido: Reloj

Que tipo de entrega desea?:
1-. Premium
2-. Regular
(Si elige premium se le dara más prioridad a su pedido al enviar)
>>> 1
Su pedido tomara prioridad sobre otros.

-- Gracias por su compra!
```

- Borrar de carrito: borramos zapatos de nuestro carrito

```
Desea comprar o borrar algo de su carrito?
1-. Comprar 2-. Borrar 3-. Salir
>>> 2

Ingrese el nombre del producto
>>> Zapatos

Producto 'Zapatos' borrado...
```

```
CARRITO

- Bolsa Roja $70

- Lentes de sol $55
```

Historial

Aquí podemos ver el historial de nuestras compras, siendo pedidos que aun no se entregan

```
HISTORIAL DE COMPRAS:

Ultimo producto comprado:
Camisa

Que desea ver?
1-. Entregados 2-. No entregados 3-. Todos 4-. Regresar
>>> 2

Productos No Entregados:
- Camisa $45
- Reloj $100
```

Conclusión

El sistema desarrollado integra múltiples estructuras de datos y algoritmos para gestionar eficientemente productos, tareas y empleados dentro de una tienda en línea.

A través de **listas**, **pilas**, **colas de prioridad**, **árboles binarios** y **HashMaps**, se implementa un control de inventario, historial de compras, asignación de tareas y jerarquía de empleados.